

For this, we use a passive TLS dataset that contains information about more than seven years of real-world TLS usage, containing more than 225 million certificates derived from more than 300 billion connections—which provides us with insights on the effect of cross-signs on real user connections. For a broad coverage of CA certificates, we combine this private, user-centered dataset with publicly available data from Certificate Transparency (CT) logs.

The main contributions of our paper are as follows:

- We provide a classification of different cross-sign patterns and analyze them with respect to their benefits, but also their risks of unexpected effects on the trust system.
- We systematically analyze the use of cross-signing in PKI systems, with a focus on the Web PKI. Thereby, we reveal problematic cross-signs that render certificate revocation or root store removals ineffective, leading to unwanted valid trust paths. We also find legit use cases, e.g., cross-signing enabled the quick tremendous success of Let’s Encrypt, eases the transition to progressive cryptography while maintaining compatibility for legacy applications, and makes a *single* certificate trusted across different applications and operating systems, achieving a non-disruptive user experience.
- We propose new rules and guidance for cross-signing to preserve its positive potential but mitigate enclosed risks.

2 BACKGROUND

This section gives a brief overview of how CAs establish trust, how trust is anchored in root stores, and how certificate revocation is applied today. For a thorough description of PKIs and their fundamental concepts, we refer readers to [28, 47].

Operating systems and some web browsers maintain root stores. They serve as trust anchors when validating certificates: To be valid, a certificate must be issued—directly or indirectly—by a trusted *root certificate*, i.e., a certificate that is included in the root store. Root certificates mostly issue *intermediate certificates* which can issue further intermediate or *leaf certificates*. Figure 1 shows an example with several root (R_i), intermediate (I_i), and leaf (L_i) certificates.

In case of breaches like stolen private keys, certificates have to be revoked. Revocation information is traditionally distributed by CAs using Certificate Revocation Lists (CRLs) [48] or interactively using OCSP [38]. However, these mechanisms mostly remain unused due to (i) the overhead for distribution and (ii) inherent privacy concerns [54, 86]. Applications thus often solely rely on the current state of the operating system’s root store.

Consequently, Browsers and operating systems have started shipping vendor-controlled CRLs. Mozilla uses *OneCRL* [39], Google *CRLSets* [80] and a blacklist [79]; Microsoft and Apple include information on blocked CA certificates in their root stores [51, 60].

3 CROSS-SIGNING

We next introduce cross-signing and provide a classification of the different cross-signing patterns used in the remainder of the paper.

3.1 Cross-signing: Definition

We illustrate cross-signing with a typical use-case: To issue trusted certificates, a CA must be included in the respective root stores of web browsers, operating systems, etc. Inclusion in these root stores

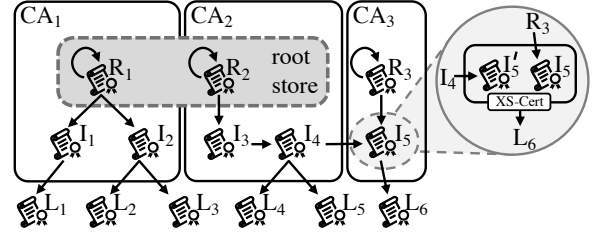


Figure 1: An example PKI including a cross-sign ($I_5 + I'_5$).

requires time demanding audit and certification processes. During the process of being included in root stores, a CA may already want to issue trusted certificates. To this end, another CA_{trusted} , whose certificate is already included in root stores, *cross-signs* the CA’s root or intermediate certificate to create a trust path that ends in the already trusted root certificate of CA_{trusted} . In Figure 1, the intermediate certificate I_5 is cross-signed by I_4 , providing a trust path to R_2 . As real-world example, Let’s Encrypt has been using an intermediate certificate cross-signed by IdenTrust to already issue certificates while waiting for root store inclusion of its own root certificate [42]. Similarly, CAs that are included in only some root stores can use cross-signing to extend trust to further root stores.

CAs typically call this *cross-signing* (also *cross-certification*) and the resulting certificates *cross-certificates* [42, 43]. Analogously, RFC 5280 defines a cross-certificate as a CA certificate that has different *entities* as issuer and subject [6]. In this paper, we use a broader definition: (i) To analyze cross-signing for *all* certificate types, i.e., root, intermediate, and leaf certificates, we consider all certificates, not only CA certificates. (ii) To also track effects of signing a certificate with multiple CA certificates of the *same* entity, we only require signatures by two different CA certificates, but not that issuer and subject are controlled by *different* entities.

Specifically, our definition is as follows (cf. Figure 1). To *cross-sign* a certificate (here: I_5) that was originally issued by R_3 , a CA certificate (here: I_4) creates and signs a copy I'_5 which has the same subject and public key as I_5 . This process is necessary as each certificate has exactly one issuer field [6], i.e., $issuer(I_5) = R_3 \neq issuer(I'_5) = I_4$. Thus, a cross-sign is a certificate for which another certificate exists that has the same subject and public key, but a different issuer and signature. These certificates form a *cross-sign certificate* (XS-Cert). The certificates of a XS-Cert can be used interchangeably: If I_5 and I'_5 are CA certificates, a certificate issued by I_5 will also validate using I'_5 . In more detail: when a certificate is validated, the validating software searches a CA certificate whose subject equals the issuer of the current certificate. It then checks that the signature of the current certificate validates against the public key of the CA certificate. As all certificates of a XS-Cert share subject and public key, I_5 and I'_5 can be used interchangeably.

Note that there is a difference between cross-signing and certificate *re-issuances without re-keying*. When a certificate reaches the end of its validity, it often is replaced by a certificate with the same subject, key, and a new validity period. We need to distinguish these cases from cross-signing. Telling them apart is complicated since cross-signs often (and legitimately) do not have the exact same validity periods as the original certificate, e.g., because the

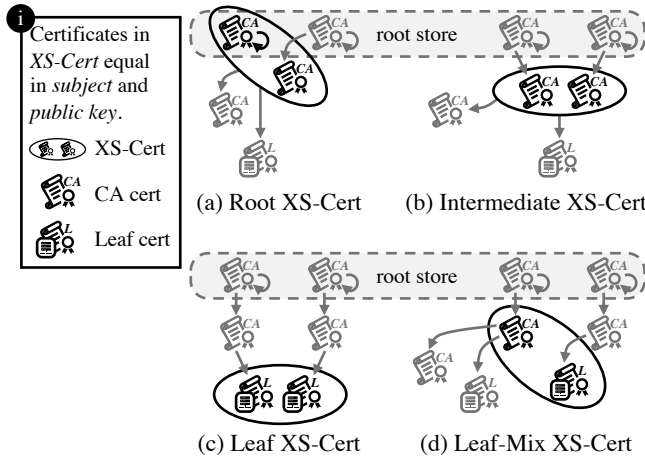


Figure 2: Cross-signing types. We observe many root, intermediate and leaf XS-Certs in our dataset. Luckily, leaf-mix XS-Certs remain a theory: We find no such problematic case.

not before field should correspond to the issuance date [68]. As, e.g., GoDaddy allows for a renewal of certificates up to 120 days prior to expiration, we require certificates to have an overlapping validity period of at least 121 days to be considered cross-signs.

3.2 Cross-signing: Classification

For our analysis of cross-signing and corresponding risks for the PKI, we classify XS-Certs into 4 types. These types depend on the type of certificates in the XS-Cert. We illustrate this in Figure 2.

Root XS-Certs. A root XS-Cert comprises at least two CA certificates of different issuers. At least one of them is part of a root store (cf. Figure 2a). When CAs use the term cross-signing, they typically refer to this type. This approach extends trust to root stores that do not include a CA’s root certificate and can extend a CA’s trust to more applications or operating systems. Likewise, an already trusted root can bootstrap trust in a new CA certificate by cross-signing it. Moreover, CAs can start to use a new CA certificate without disrupting compatibility with old applications that do not receive root store updates. However, as we will discuss later, such cross-signs increase the complexity of removing trust in certificates that are part of a root store. Trust paths can inadvertently remain valid via paths using other certificates of the XS-Cert (Section 5.1).

Intermediate XS-Certs. Intermediate XS-Certs contain two or more intermediate CA certificates from different issuers, but no root store certificate (cf. Figure 2b). They are similar to root XS-Certs, but no certificate in the XS-Cert is directly trusted by root store maintainers. The same benefits (bootstrapping, large trust coverage, and compatibility with old applications) apply. Additionally, CAs without an own root certificate may employ such cross-signing to root trust in multiple CAs, and thus achieve independence from a single business partner. However, intermediate XS-Certs also are problematic if certificate revocations are not thoroughly applied to all certificates of a XS-Cert (cf. Section 5.1).

Leaf XS-Certs. Leaf XS-Certs contain only leaf certificates (cf. Figure 2c). This could in principle be used in cases where some client

applications only trust divergent CAs; Here, the server can send *that* certificate of which it knows that it will be trusted by the client, e.g., using the user agent data in an HTTP request to determine the client’s root store. As leaf certificates cannot issue certificates, problematic cases are limited in scope. Still, for a revocation in case of a security incident, all certificates of the XS-Cert have to be revoked – otherwise insecure certificates remain valid.

Leaf-Mix XS-Certs (Theoretical). In theory, a XS-Cert could contain leaf certificates and (root or intermediate) CA certificates (cf. Figure 2d). Private keys of CA certificates have special protection requirements such as the use of sealed hardware. As private keys of leaf certificates are typically present on comparably vulnerable end-systems, a leaf-mix XS-Cert would put a key that can issue certificates at high risk. Fortunately, our dataset does not contain any valid leaf-mix XS-Certs.

4 DATASETS AND METHODOLOGY

The analysis in this paper is based on a dataset of certificates used in the wild. The certificates are passively collected from the outgoing SSL/TLS connections on all ports of several universities and research networks mainly located in North America. Beyond public data (e.g., CT Logs), it also contains certificates that are not publicly available (e.g., user or private certificates) and thereby enables us to take a broad yet unseen perspective on cross-signing. Our dataset spans a period of more than 7 years, starting February 2012 and ending August 2019, covering more than 300 billion TLS connections. For a broad view on CA certificates, we extend this dataset with CA certificates from CT logs² (those logs used by crt.sh [58]).

The passive data collection effort was cleared by the respective responsible party at every contributing institution. Our collection effort focuses on machine to machine communication and excludes or anonymizes sensitive information. See Appendix A for details.

We release the toolchain [46] that enables the analysis of the cross-signing relationships of X.509 certificates from any source (e.g., Censys [33] or CT logs [58]). Since that part of our data set derived from passive measurements is subject to NDAs and contains private certificates that are not contained in public repositories, we are unable to release our data set. Our toolchain can, however, be used to reproduce our analysis using other data sources, e.g., CT.

We validate certificates using our custom validation logic which closely mirrors the way browsers perform certificate validation. We build potential paths using name-matching between subject and issuer fields. Unlike browsers, we build all possible paths from a certificate to root certificates³. We do not just search for the shortest possible path. To this end, we check each path separately by passing the complete path to OpenSSL for validation and using only the specific root certificate for this path as trust anchor. We obtain the earliest and latest possible validity dates based on the *not before* and *not after* dates of the path’s certificates and use these for the validation. We also check if a path is valid given the path-length-constraints of its CA certificates. To obtain validity information for

²These contain 156,315 CA certificates, however, 147,439 of them are used for *Google Certificate Transparency (Precert Signing)* which we exclude from our analysis. Further 2,635 are already included in our passive dataset. CT thus adds 6,241 CA certificates.

³Due to computational complexity, we base our evaluation for some CT certificates on all paths of length 12 or smaller (multiple million paths per certificate); for the vast majority of certificates, also longer paths were validated and used.

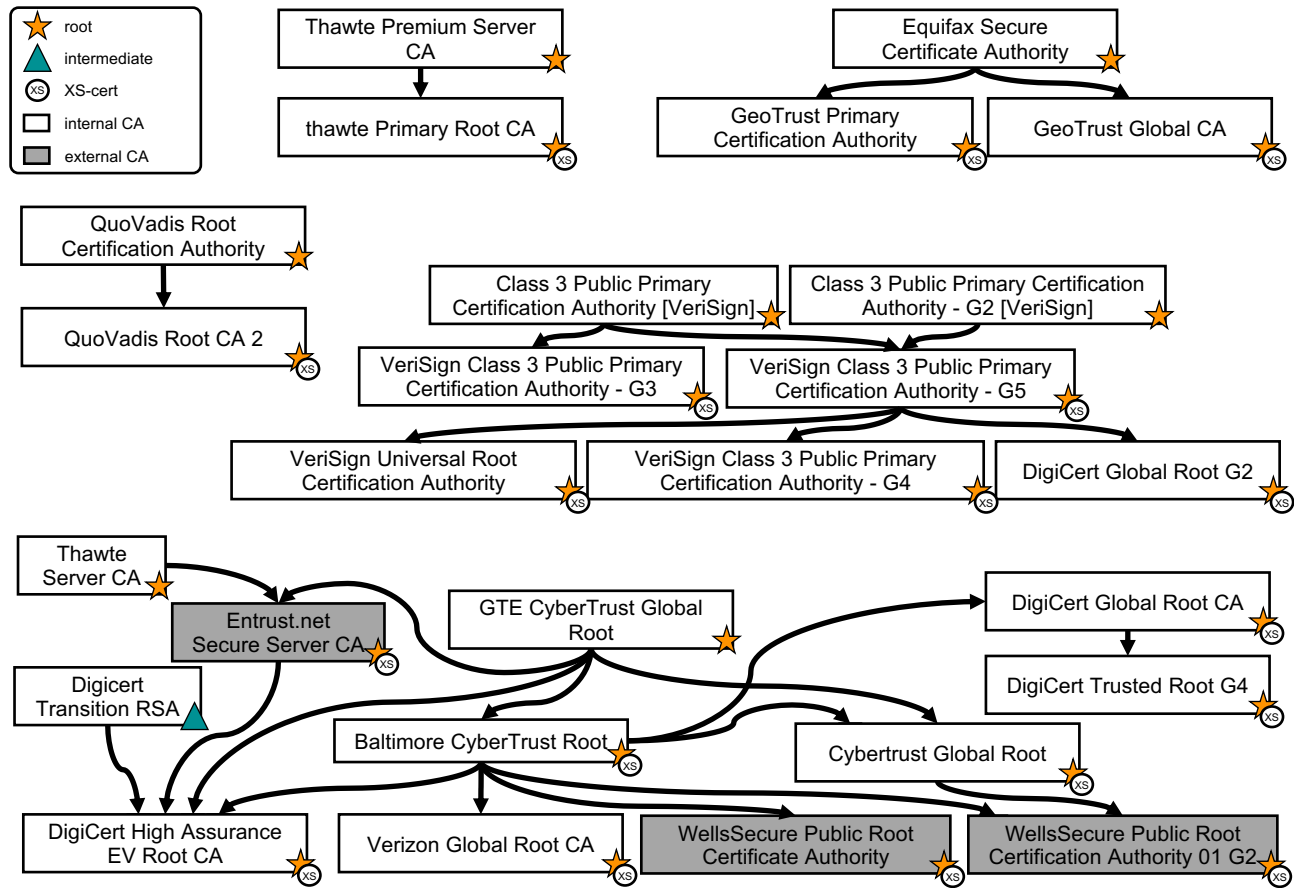


Figure 5: Root XS-Certs of DigiCert and its subsidiaries: Cross-signs created prior to acquisition by DigiCert form islands.

Global Root CA and *DigiCert High Assurance EV Root CA*. The latter was additionally cross-signed by *GTE CyberTrust Global Root* and *DigiCert Transition RSA Root* (and *Entrust.net Secure Server Certification Authority*; cf. Section 5.6.1). However, more prevalent are the cross-signs of external organizations. First, Verizon started cross-signing root certificates of Wells Fargo since 2013. Specifically the *Baltimore CyberTrust Root* cross-signed Wells Fargo's *WellsSecure Public Root Certificate Authority* and *WellsSecure Public Root Certificate Authority 01 G2* in 2013 and 2015, respectively. The latter was already cross-signed by *Verizon Global Root CA* in 2013. All corresponding intermediates were revoked by the CA's CRL and Mozilla's OneCRL in 2017, when the roots were removed from almost all root stores (after request by Wells Fargo [15]), except for Apple's store which still includes *WellsSecure Public Root Certificate Authority*. Similarly, the Verizon-controlled *GTE CyberTrust Global Root* cross-signed *Certipost E-Trust Primary Normalized CA* expanding the Microsoft-only trust to a broad trust coverage and creating another cross-sign that survived the acquisition by DigiCert.

Verizon also cross-signed several state-controlled CAs which is potentially problematic (cf. Section 5.2). In 2010 and 2013, *Baltimore CyberTrust Root* cross-signed the *Swiss Government root* which increased the formerly limited trust (Apple and Microsoft only). One of the cross-signs expired in 2014, the other stayed active till

after the DigiCert acquisition. In 2013, *Belgium Root CA2* replaced a cross-sign by (non-DigiCert) *GlobalSign Root* from 2007 with a further Verizon cross-sign by *CyberTrust Global Root*. This cross-sign also stayed active after the DigiCert acquisition until October 2017 when it was revoked in OneCRL [19]. Verizon also used *GTE CyberTrust Global Root* and *Baltimore CyberTrust Root* to cross-sign Portugal's *SCEE ECRaizEstado* (formerly trusted by Microsoft only). This time, DigiCert actively repeated the latter cross-sign shortly after obtaining control over the CyberTrust roots. However, when the cross-sign by *GTE CyberTrust Global Root* expired in August 2018, the other cross-signs were revoked due to a series of misissuances [21]. Thus, several CAs provided state-controlled CAs with a larger trust coverage than provided by the original roots.

Finally, we find a cross-sign between DigiCert's very own roots, i.e., *DigiCert Global Root CA* cross-signed *DigiCert Trusted Root G4*.

B.7 Cross-Signing in the Grid PKI

In the grid-PKI, we do not find a real XS-Cert, but an interesting re-issuance. Originally, *ESnet Root CA 1* issued an intermediate named *NERSC Online CA*. A month before its expiry, NERSC created a corresponding root certificate instead of requesting a new intermediate. One could argue that ESnet helped NERSC to bootstrap the trust in its later root.